Embedded Systems + Semiconductor Roadmap — ELITE Edition

DV-first + C-first | Top-10 US MS (IC Design) Optimized | Funded-MS Competitive

Student: University of Regina Start date: **2026-02-19** Time commitment: **~32–38 hours/week** (upgraded from 25–30) Primary OS: **Windows 10 + WSL2 (Ubuntu)** Primary target: **Funded MS or direct PhD in IC Design / Semiconductor — Top-10 US** Secondary target: **Embedded Systems (C-first, production-grade firmware)** Roadmap version: ELITE v1 (upgraded from Version 8)

Why This Version Is Different

The original roadmap gets you to top-20 to top-30 admission with some funding chance. **This version targets top-10 admission with funded MS or PhD offer.** The differences:

Dimension	Original	ELITE
Hours/week	25–30	32–38
Doc reading	Vague	Exact section + page target per week
Review system	None	Explicit weekly review ritual (30 min)
Research	Starts Week 4 (loose)	Starts Week 1 (structured, publication-targeted)
Paper target	Workshop-level draft	IEEE SSCS Student Paper Contest / DAC SRC
HDLBits	Streak only	Streak + written explanation of every solved problem
RA pipeline	Starts Week 20	Starts Week 6 (aggressive early contact)
Professor cold-email	Week 20	Week 8 (US professors included)
Flagship repo quality	Good README	Reproducible artifact: exact commands, CI badge, waveform gallery
C depth	Surface	MISRA-aware, defensive, lint-clean

Non-Negotiable Weekly Habits (Every Single Week)

These are **fixed rituals** — no exceptions, even during hard weeks.

Habit	Time	Purpose
DV drill (TB improvement, randomized test, or assertion note)	30–45 min	Core DV identity
C drill (1 new program or refactor in WSL2)	30 min	Embedded identity

Habit	Time	Purpose
HDLBits (minimum 5 problems; write 1-line explanation per problem)	45–60 min	RTL fluency
Weekly review ritual (see below)	30 min	Retention + tracking
Job/professor sweep (5–10 roles saved, 2 messages sent)	45 min	Pipeline never goes cold
Research log entry (even 3 sentences)	15 min	Evidence of research habit
Git push (at least 1 meaningful commit)	—	Portfolio discipline

Weekly Review Ritual (every Sunday, 30 min)

Open a single doc: `weeks/week-XX/weekly-review.md`

Answer these 7 questions every week:

1. What did I build this week? (1–3 bullets with commit links)
2. What did I read? (doc section + 1-sentence summary each)
3. What concept confused me most? (name it exactly)
4. What HDLBits problem challenged me most? (paste your explanation)
5. What did I push that I'm proud of?
6. What is my one research log entry this week?
7. What are my top 3 goals for next week?

This review is the **single most important habit** for long-term retention.

Toolchain (unchanged but with version pins)

Stage 0 — Week 1: Windows essentials

- Git for Windows: https://git-scm.com/download/win (pin version in your `notes/toolchain.md`)
- VS Code: https://code.visualstudio.com/
- Python 3.11+: https://www.python.org/downloads/windows/
- GitHub CLI: https://cli.github.com/

Stage 1 — Weeks 1–2: WSL2 + C foundations

- WSL2: https://learn.microsoft.com/windows/wsl/install
- In WSL2: `sudo apt install build-essential gdb valgrind cppcheck python3 python3-pip`
- Add `clang-format` and `clang-tidy` (Week 3+): `sudo apt install clang-format clang-tidy`

Stage 2 — Weeks 1–2: RTL/DV toolchain

- Icarus Verilog (WSL2): `sudo apt install iverilog`
- GTKWave (WSL2): `sudo apt install gtkwave`
- Verilator (build from source, Week 11): https://www.verilator.org/guide/latest/install.html

Stage 3 — Weeks 4–6: Embedded hardware

Same as original. Buy Pico + BME280 + logic analyzer in Phase 1.

GitHub Structure (ELITE — recruiter + professor + MS admissions quality)

Repos

```
embedded-journey/          # Hub repo — landing page + weekly logs
  README.md                # Professional landing page (always updated)
  weeks/
    week-01/ ... week-34/
      weekly-plan.md
      weekly-report.md
      weekly-review.md     # NEW: 7-question review ritual
      rtl-dv/
      embedded-c/
      reading/             # NEW: doc reading notes per week
  notes/
  reading-notes/
    papers/                # NEW: one .md per paper
    docs/                  # NEW: doc section summaries
  paper/                   # Research replication study
  resume/
  professors/
  internship/

rtl-uart-tx/               # Flagship DV repo (must be publishable-quality)
  README.md                # Full methodology + results + waveform gallery
  rtl/
  tb/
  scripts/
    run_tests.py
    coverage_report.py     # NEW
  docs/
    bug-diary.md
    waveform-gallery/      # PNG screenshots of key waveforms
    test-plan.md
    results-summary.md     # NEW: quantitative results table
  .github/
    workflows/
      ci.yml               # NEW: GitHub Actions CI (iverilog regression)

pico-data-logger/          # Embedded project repo
  README.md
  src/
  include/
  docs/
  assets/                  # Photos, waveform captures from logic analyzer
```

Quality Bar (ELITE level — above original)

Every "judgeable" repo must additionally have:

- **GitHub Actions CI badge** in README (green = passing tests automatically)
- **Results table** (quantitative: N tests, N bugs caught, catch rate %)

- **Waveform gallery** (`docs/waveform-gallery/`) with labeled screenshots
- **Exact tool versions** in `notes/toolchain.md`
- **Bug diary** with root cause analysis (not just "bug fixed")
- **Architecture diagram** (even hand-drawn + scanned, or ASCII art)
- All commands tested on a clean WSL2 install (reproducibility guarantee)

Master Reading List With Exact Sections Per Week

HOW TO USE THIS LIST

- Column "Read this week" = new material
- Column "Review this week" = re-read or quiz yourself (spaced repetition)
- Do NOT read ahead. Do NOT skip review weeks.

Tier 1: MUST (non-negotiable)

A. Pro Git — https://git-scm.com/book/en/v2

Week	Read this week	Review this week
1	Ch 1 (Getting Started): all — ~20 pages	—
1	Ch 2 (Git Basics): §2.1–2.4 — recording, history, undoing	—
2	Ch 2 §2.5–2.7 — remotes, tagging, aliases	Ch 1 all
2	Ch 3 §3.1–3.3 — branches, new branch, switching	Ch 2 §2.1–2.4
9	Ch 3 §3.4–3.6 — rebasing, remote branches	Ch 3 §3.1–3.3
11	Ch 5 §5.1–5.2 — distributed workflows	Ch 3 §3.4–3.6
24	Ch 8 §8.1–8.3 — Git hooks (for CI awareness)	Ch 5 §5.1–5.2

B. Missing Semester — https://missing.csail.mit.edu/

Week	Watch/Read this week	Review this week
1	Lecture 1: Course overview + shell basics (full)	—
1	Lecture 2: Shell tools + scripting (full)	—
2	Lecture 3: Editors (Vim) — §overview + basic motions	Lecture 1
2	Lecture 4: Data wrangling (grep, sed, awk)	Lecture 2
11	Lecture 6: Version control (Git) deep dive	Lectures 3–4
11	Lecture 9: Security + cryptography (awareness)	—

C. Icarus Verilog docs — https://steveicarus.github.io/iverilog/

Week	Read this week	Review this week
2	"Getting Started" page: compilation flags (`-o`, `vvp`, `-Wall`)	—
2	"Command line options" page: all flags, focus on `-g2012`	—
9	"System tasks" section: `$display`, `$monitor`, `$dumpfile`, `$dumpvars`, `$finish`, `$stop`	Week 2 flags
11	"Compiler directives" section: `` `timescale ``, `` `define ``, `` `include ``	Week 9 system tasks

Week	Read this week	Review this week
23	"Known issues / limitations" page — important for debugging	All prior iverilog reading

D. GTKWave — https://gtkwave.sourceforge.net/

Week	Read this week	Review this week
2	GTKWave "User Guide" §1–2: launching, loading VCD, adding signals	—
2	GTKWave "User Guide" §3: zoom, cursors, time markers	—
9	GTKWave "User Guide" §4: signal grouping, color assignment	Week 2 §1–3
10	GTKWave "User Guide" §5: saving/restoring signal layout (`.gtkw` save files)	§4
14	GTKWave "User Guide" §6: searching for signal edges, markers for protocol decode	§5

GTKWave guide PDF: http://gtkwave.sourceforge.net/gtkwave.pdf Read it in the PDF — it is more complete than the website.

E. HDLBits — https://hdlbits.01xz.net/wiki/Main_Page

Not a doc to read — a problem set to DO. But use this structured progression:

Week	Problem set to complete	Write-up requirement
2	Getting Started + Wires section (all ~8 problems)	1-line explanation per problem in `weeks/week-02/hdlbits-notes.md`
3	Gates section (all ~7) + Vectors section (all ~12)	Same
4	Modules section (all ~6) + Procedures (all ~10)	Same
5	More Verilog features (all ~5) + Counters (all ~7)	Same
6	Shift registers (all ~9)	Same
7	Finite state machines §1 (Mealy/Moore, all ~10)	Same
8	Finite state machines §2 (serial receiver, all ~8)	Same
9–34	Minimum 5 problems/week from any unsolved section	Same (mandatory)
Monthly	Re-do 3 problems you got wrong from the prior month	Note what changed

F. Raspberry Pi Pico C/C++ SDK docs

Main hub: https://www.raspberrypi.com/documentation/microcontrollers/ SDK PDF: https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf

Week	Read this week (PDF chapter or online page)	Review this week
6	SDK PDF §1: Introduction + §2: Quick start (all)	—
6	SDK PDF §3: Library architecture overview (skim)	—
7	SDK PDF §4: hardware_gpio — gpio_init, gpio_set_dir, gpio_put, gpio_get	§1–2
7	SDK PDF §5: hardware_timer — time_us_64, add_repeating_timer_ms	§4
8	SDK PDF §6: hardware_i2c — i2c_init, i2c_write_blocking, i2c_read_blocking	§5
14	SDK PDF §7: hardware_uart — uart_init, uart_puts, uart_getc	§6
15	Sensor datasheet (BME280 or MPU-6050) — register map section (full)	§7
16	SDK PDF §8: hardware_spi (if using SPI sensor)	Sensor datasheet
17	Pico datasheet §4: power, GPIO electrical specs, UART pin mux	SDK §7

G. Keshav "How to Read a Paper" PDF

https://web.stanford.edu/class/ee384m/Handouts/HowtoReadPaper.pdf

Week	Action
4	Read fully (6 pages, 20 min). Take handwritten notes on the 3-pass method.
13	Re-read before building your reading matrix.
26	Re-read before writing your paper draft introduction.

Tier 2: HIGH-VALUE (complete at least 80%)

H. Learn-C — https://www.learn-c.org/

Week	Complete these tutorials	Review
1	Hello World, Variables, Arrays, Strings, String Format	—
2	While loops, For loops, Functions, Static	Week 1 set
3	Pointers, Structures, Function arguments by reference	Week 2 set
4	Dynamic allocation, Arrays + Pointers, Linked lists	Week 3 set
5	Unions, Pointer arithmetic, Recursion, Static variables	Week 4 set

After Week 5, Learn-C is complete. Move to Beej's guide for depth.

I. Beej's C Guide — https://beej.us/guide/bgc/

Week	Read this week	Review
6	Ch 1–5: types, variables, flow control	Learn-C weeks 1–5
7	Ch 6–9: functions, arrays, strings, structs	Ch 1–5
8	Ch 12–14: pointers, manual memory mgmt	Ch 6–9
14	Ch 20–22: bitwise ops, practice problems	Ch 12–14
17	Ch 23–25: error handling, signal handling	Ch 20–22

J. Embedded Artistry Blog — https://embeddedartistry.com/blog/

Do NOT read randomly. Use this structured reading list:

Week	Article to read	Why now
2	"Choosing the right volatile in C"	You're learning volatile in C
7	"Building a super loop"	You're building the Pico non-blocking scheduler
7	"Circular buffer (ring buffer) in C"	Foundational pattern
8	"A philosophy of software design for embedded systems"	RA-level thinking
14	"Error handling in C: return codes"	You're building UART CLI
17	"Timeouts and retries in embedded systems"	You're building fault state machine
19	"Writing defensive code"	DV mindset applied to C
25	"Refactoring embedded software"	Portfolio polish

K. Nand2Tetris — https://www.nand2tetris.org/

Week	Complete	Notes
3	Project 1: Boolean Logic (all gates from NAND)	Spend 2 hrs; this is RTL thinking in hardware
3	Read Ch 1 of the book (companion, free online excerpt)	
4	Project 2: Boolean Arithmetic (ALU)	2 hrs
12	Project 3: Sequential Logic (DFF, register, RAM)	Connects to your FIFO week

---

Tier 3: RESEARCH-LEVEL (for top-10 / funded positioning)

L. IEEE Std 1364-2005 (Verilog LRM) — free via IEEE Xplore with university login

Week	What to read	Why
9	§1 (scope) + §2 (overview) — 20 pages	Understand the standard, not just examples
12	§12: Behavioral modeling (always, initial, blocking vs non-blocking)	Deep RTL correctness
23	§14: Specify blocks + timing checks	CDC + timing awareness

M. "Writing Science" by Joshua Schimel (book — buy or library)

Week	Read	Why
13	Ch 1–3: story structure of papers	Foundational for your paper
22	Ch 4–6: OCAR framework (Opening, Challenge, Action, Resolution)	Apply to your paper structure

Week	Read	Why
26	Ch 7–9: writing sentences that work	Apply to your intro/abstract

N. Google Scholar + Paper Reading Habit

Week	Action
4	Find 3 papers on "design verification UART" or "randomized testing RTL" — save to Zotero
8	Read 1 paper fully using 3-pass method — write `reading-notes/paper-01.md`
12	Read 1 paper — `reading-notes/paper-02.md`
13	Build `paper/related-work-matrix.csv` (10 papers minimum)
16	Read 1 paper — write comparative note vs your method
20	Read 1 paper — write "what I would do differently" note
24	Read 1 paper — connect to your results
28	Re-read your top 3 papers with final eyes

Target paper venues to search:

- IEEE Transactions on Computer-Aided Design (TCAD)
- Design Automation Conference (DAC)
- International Test Conference (ITC)
- IEEE SSCS Student Design Contest proceedings

WEEK-BY-WEEK ROADMAP (34 Weeks) — ELITE Edition

Legend:

- □ IC/DV priority
- ⚙ Embedded (C-first)
- □ Research (evidence for MS SOP + letters)
- □ Career/RA/MS pipeline
- □ Read (specific doc section)
- □ Review (spaced repetition)

Hours/week target: **32–38 hrs**

Week 1 — Setup + Git + C refresh + Research mindset begins

Hours target: 33 hrs

Block	Task	Hours
Setup	Git for Windows + VS Code + WSL2 install	3.0
Setup	WSL2: install build-essential, gdb, valgrind, cppcheck, clang-format	1.5
Setup	Create all GitHub repos: `embedded-journey`, start `README.md` scaffold	1.5

Block	Task	Hours
C	Learn-C tutorials: Hello World, Variables, Arrays, Strings, String Format	1.5
C	Write 5 C programs in WSL2: hello, types demo, array ops, string ops, format	2.0
Git	Set up full folder structure: `weeks/`, `notes/`, `paper/`, `professors/`, `internship/`	1.5
Git	Create all CSV trackers (with headers, empty)	0.5
Career	`internship/regina-market-map.md` — 15 employers minimum (not 10)	1.5
Career	`professors/uofr-lab-map.md` — 5 professors minimum, 2 projects each	1.5
Research	**NEW:** Create `paper/research-seed.md` — write 3 candidate research questions	0.5
DV	Read: what is a testbench? (HDLBits wiki intro page)	0.5

 Read this week:

- Pro Git Ch 1 (all) + Ch 2 §2.1–2.4
- Missing Semester Lecture 1 (full) + Lecture 2 (full)
- Learn-C: 5 tutorials (Hello World → String Format)

 Review this week: nothing (Week 1 is all new)

Checkpoint:

- C compiles + runs in WSL2
- Git pushes work cleanly with meaningful commit messages
- All CSV trackers exist with headers
- `paper/research-seed.md` exists

Push:

- `weeks/week-01/weekly-plan.md`, `weekly-report.md`, `weekly-review.md`
- `weeks/week-01/embedded-c/` (5 programs)
- `internship/regina-market-map.md` (15+ entries)
- `professors/uofr-lab-map.md` (5 profs)
- `paper/research-seed.md`
- `notes/toolchain.md` (exact versions of everything installed)

Week 2 — DV toolchain + bitwise C + first self-checking TB + waveform proof

Hours target: 35 hrs

Block	Task	Hours
DV	Install iverilog + GTKWave in WSL2; verify both work	1.5
DV	Write your first DUT: 4-bit adder in Verilog	1.0
DV	Write self-checking TB: PASS/FAIL, `$display`, `$dumpfile`	2.0
DV	Open waveform in GTKWave, add all signals, zoom to see glitches	1.0
DV	HDLBits: Getting Started + Wires section (all ~8 problems)	1.5
DV	Write `weeks/week-02/hdlbits-notes.md` — 1-line explanation per problem	0.5
C	Learn-C: While loops, For loops, Functions, Static	1.0
C	Write bitwise C programs: masks, shifts, error code pattern, volatile demo	2.0

Block	Task	Hours
C	Read Embedded Artistry: "Choosing the right volatile in C"	0.5
Career	Expand Regina map: 20–25 entries	1.0
Research	Refine 1 research question from `paper/research-seed.md`	0.5

 Read this week:

- Pro Git Ch 2 §2.5–2.7 + Ch 3 §3.1–3.3
- Missing Semester Lecture 3 (Vim basics) + Lecture 4 (data wrangling)
- iverilog docs: "Getting Started" page + "Command line options" page (all flags)
- GTKWave User Guide PDF §1–3 (load, add signals, zoom, cursors)

 Review this week:

- Pro Git Ch 1 (re-read headings + re-do any git commands you're shaky on)
- Missing Semester Lecture 1 (re-watch if anything was unclear)

Checkpoint:

- TB runs and prints PASS
- VCD waveform opens in GTKWave and all signals are visible
- Can explain `$dumpfile` + `$dumpvars` + `$finish` in your own words

Push:

- `weeks/week-02/rtl-dv/adder_tb/` (DUT + TB + VCD + waveform screenshot)
- `weeks/week-02/embedded-c/bitwise/` (3+ programs)
- `weeks/week-02/hdlbits-notes.md`
- `weeks/week-02/reading/iverilog-notes.md` (your notes on the docs you read)
- `notes/toolchain.md` updated

Week 3 — Digital logic deep + FSM thinking + Nand2Tetris Project 1 + C pointers

Hours target: 35 hrs

Block	Task	Hours
DV	HDLBits: Gates section (all ~7) + Vectors section (all ~12)	2.5
DV	HDLBits notes: 1-line explanation each	0.5
DV	Write a Verilog FSM (traffic light or vending machine): encode states, draw state diagram first	2.0
DV	Write self-checking TB for FSM: cover all state transitions	2.0
DV	Nand2Tetris Project 1: implement all basic gates from NAND only	2.0
DV	Read Nand2Tetris Ch 1 (book excerpt or website)	0.5
C	Learn-C: Pointers, Structures, Function arguments by reference	1.0
C	C FSM simulation: implement the same FSM as your Verilog version in C	2.0
Career	Professors: grow `uofr-lab-map.md` to 8 professors, 2 projects each	1.5
Career	Job sweep: add 10 Canada-wide semiconductor roles to `target-companies.csv`	1.0

 Read this week:

- Learn-C: Pointers, Structures, Function by reference (3 tutorials)
- Nand2Tetris website "Projects" page for Project 1 specification (full)

 Review this week:

- Pro Git Ch 3 §3.1–3.3 (quiz yourself: what is a merge conflict? how do you rebase?)
- HDLBits: re-do 2 problems from Week 2 that you found hardest

Checkpoint:

- FSM module + TB passes all state transitions
- Nand2Tetris Project 1 complete (all gates built and tested)
- C FSM version compiles and matches Verilog FSM behavior

Push:

- `weeks/week-03/rtl-dv/fsm_traffic_light/` (FSM RTL + TB + waveform screenshot)
- `weeks/week-03/embedded-c/fsm-c/` (C version of FSM)
- `weeks/week-03/hdlbits-notes.md`
- `notes/week-03-fsm-states.md` (your state diagram as ASCII art)
- `professors/uofr-lab-map.md` updated

Week 4 — Scoreboard pattern + research formally starts + Nand2Tetris Project 2 + dynamic C

Hours target: 36 hrs

Block	Task	Hours
DV	HDLBits: Modules section (all ~6) + Procedures section (all ~10)	2.5
DV	HDLBits notes	0.5
DV	Build scoreboard: expected queue + actual compare + failure reporter	2.5
DV	Inject 2 deliberate bugs — confirm TB catches both, record in bug diary	1.0
DV	Nand2Tetris Project 2: ALU implementation	2.0
C	Learn-C: Dynamic allocation, Arrays+Pointers, Linked lists	1.5
C	Implement a simple scoreboard in C (queue + compare) — mirrors your DV scoreboard	2.0
Research	**Formally start:** Read Keshav PDF (20 min); take handwritten notes	0.5
Research	Finalize `paper/research-question.md` — ONE clear question, not 3	0.5
Research	Create `paper/research-log.md` + `paper/method-notes.md` + `paper/related-work-matrix.csv`	0.5
Research	Google Scholar: find 3 papers on "RTL verification" or "random testing hardware" — save to Zotero	1.0
Career	Add 10 Canada-wide targets to `target-companies.csv` (tag: DV / RTL / FPGA)	0.5

 Read this week:

- Learn-C: Dynamic allocation, Linked lists (3 tutorials)
- Keshav "How to Read a Paper" PDF (all 6 pages — read twice)

 Review this week:

- Week 2 TB structure: can you re-write the adder TB from memory in 10 minutes? Try it.
- Learn-C Weeks 1–3: skim your programs; fix any that use bad style

Checkpoint:

- Scoreboard module works, 2 bugs caught, failure message includes test name
- `paper/` directory has 4 files (question, log, method, matrix)
- 3 papers saved in Zotero or `reading-notes/papers/`

Push:

- `weeks/week-04/rtl-dv/scoreboard_exercise/`
- `weeks/week-04/embedded-c/scoreboard-c/`
- `weeks/week-04/hdlbits-notes.md`
- `paper/research-question.md`, `paper/research-log.md`, `paper/method-notes.md`
- `paper/related-work-matrix.csv` (3 entries seeded)

Week 5 — UART protocol mastery + spec writing + More Verilog features + C unions/recursion

Hours target: 35 hrs

Block	Task	Hours
DV	HDLBits: More Verilog features (all ~5) + Counters (all ~7)	2.0
DV	HDLBits notes	0.5
DV	Study UART frame: start bit, data bits, parity, stop bit, baud math	1.5
DV	Write `uart_spec.md` — FULL specification (frame, timing, edge cases, error cases)	2.0
DV	Write `test_plan.md` — at least 15 test cases (not 5); categorize: smoke / corner / negative	1.5
C	Learn-C: Unions, Pointer arithmetic, Recursion, Static variables	1.5
C	C UART frame simulator: encode a byte to UART bit stream in C	2.0
Research	`paper/research-log.md`: write paragraph explaining how UART will be your experimental vehicle	0.5
Research	Read paper #1 using 3-pass method — write `reading-notes/papers/paper-01.md`	1.5
Career	Networking: message 2 people (1 UofR alumni + 1 Canada DV/FPGA engineer on LinkedIn)	0.5

 Read this week:

- Learn-C: Unions, Pointer arithmetic, Recursion, Static (4 tutorials)
- UART Wikipedia article (read fully for frame structure + timing)
- YouTube: watch 1 video "UART protocol explained" (≥15 min) — take bullet notes

 Review this week:

- Week 3 FSM: draw your traffic light state diagram from memory, check against your notes
- Scoreboard pattern (Week 4): explain it to yourself out loud in 3 minutes

Checkpoint:

- `uart_spec.md` is complete: covers start/stop bits, baud rate formula, 8N1 frame, error cases
- `test_plan.md` has 15+ test cases in a table
- Paper #1 summary written

Push:

- `weeks/week-05/rtl-dv/uart-spec/uart_spec.md`
- `weeks/week-05/rtl-dv/uart-spec/test_plan.md`
- `weeks/week-05/embedded-c/uart-frame-sim/`
- `reading-notes/papers/paper-01.md`
- `internship/networking-tracker.csv` updated

Week 6 — Pico bring-up + hardware first flash + Shift Registers HDLBits + RA pipeline STARTS

Hours target: 36 hrs

Block	Task	Hours
Hardware	Buy Pico + BME280 + breadboard + cables (order if not yet arrived)	—
Hardware	Install Pico C/C++ SDK in WSL2 — follow SDK Quick Start fully	2.5
Hardware	Build + flash blink example; then UART "hello" banner	2.0
Hardware	Serial monitor: use `minicom` or VS Code serial port extension	0.5
DV	HDLBits: Shift registers section (all ~9 problems)	2.0
DV	HDLBits notes	0.5
DV	Keep DV habit: improve adder TB — add 3 more edge cases	1.0
C	Beej's C Guide Ch 1–5 (types, variables, flow control)	1.5
Research	`paper/research-log.md` entry: describe what experimental measurements you will take	0.5
Career	**RA PIPELINE STARTS:** Email 1 UofR professor — short intro, link to your GitHub, ask to meet	1.0
Career	Job sweep: 5–10 roles saved	0.5

 Read this week:

- Pico SDK PDF §1: Introduction + §2: Quick Start (all)
- Pico SDK PDF §3: Library architecture overview (skim, 30 min)
- Beej's C Guide Ch 1–5

 Review this week:

- iverilog command flags (Week 2): run `iverilog --help` and explain each flag you know
- GTKWave §1–3 (Week 2): open your adder VCD and add a new signal you haven't shown before

Checkpoint:

- Pico flashed, prints UART banner with your name + date + firmware version
- Banner is visible in serial monitor with exact baud rate noted in `toolchain-setup.md`

- RA email #1 drafted or sent

Push:

- `weeks/week-06/toolchain-setup.md` (exact SDK version, cmake flags, all commands)
- `weeks/week-06/pico-hello-uart/` (source + log screenshot)
- `weeks/week-06/hdlbits-notes.md`
- `professors/outreach-tracker.csv` updated
- `internship/applications-tracker.csv` updated (apply to 2 roles this week)

Week 7 — Pico GPIO + non-blocking scheduler + FSM HDLBits §1 + Beej Ch 6–9

Hours target: 36 hrs

Block	Task	Hours
Hardware	GPIO: blink with timer-based non-blocking scheduler (no `sleep_ms` in main loop)	2.5
Hardware	Add timeout + error code pattern to scheduler	1.5
Hardware	Test: scheduler runs 10 tasks correctly for 10 minutes — log proof	1.0
DV	HDLBits: FSM §1 (Mealy/Moore, all ~10 problems)	2.5
DV	HDLBits notes	0.5
DV	DV habit: write 1 TB with at least 5 test cases (any module)	1.5
C	Beej's C Guide Ch 6–9 (functions, arrays, strings, structs)	1.5
C	Read Embedded Artistry: "Building a super loop" + "Circular buffer in C"	1.0
Research	`paper/research-log.md` entry	0.3
Career	Add 2 professors to `uofr-lab-map.md` (total 10 now); add 1 abstract note each	1.0
Career	Networking: 2 messages	0.5

 Read this week:

- Pico SDK PDF §4: hardware_gpio (all functions in the section)
- Pico SDK PDF §5: hardware_timer (time_us_64, add_repeating_timer_ms)
- Beej's C Guide Ch 6–9
- Embedded Artistry: "Building a super loop" + "Circular buffer in C"

 Review this week:

- UART spec (Week 5): read your `uart_spec.md` — does it cover parity error? baud rate mismatch? If not, update it.
- HDLBits Week 6 problems: re-do the 2 hardest shift register problems

Checkpoint:

- Pico runs non-blocking scheduler for 10 minutes — no blocking calls in main loop
- All tasks have timeouts and return error codes (not void)
- `professors/uofr-lab-map.md` has 10 professors

Push:

- `weeks/week-07/pico-scheduler/` (source + 10-min log file)
- `weeks/week-07/rtl-dv/multi_test_tb/`
- `weeks/week-07/hdlbits-notes.md`
- `professors/uofr-lab-map.md` updated

Week 8 — I2C planning + constrained random intro + FSM HDLBits §2 + US professor emails STARTS

Hours target: 36 hrs

Block	Task	Hours
Hardware	I2C study: read BME280 datasheet §4 (register map) + Pico I2C section	1.5
Hardware	Write I2C driver skeleton: init + WHO_AM_I read + error return	2.0
Hardware	Wire BME280 to Pico: SDA, SCL, VCC, GND — draw wiring diagram	0.5
DV	HDLBits: FSM §2 (serial receiver, all ~8 problems)	2.0
DV	HDLBits notes	0.5
DV	Constrained random: write TB that generates random 8-bit values, feeds to adder, scoreboard compares	2.0
C	Beej's Ch 12–14 (pointers, manual memory management)	1.5
Research	Google Scholar: find 5 more papers (now 8 total) — update matrix	1.0
Career	Canada-wide sweep: add 10 more targets to `target-companies.csv`	0.5
Career	**US PROFESSOR PIPELINE STARTS:** Identify 3 target MS/PhD programs — find 2 professors per school	1.0
Career	Draft `professors/meeting-request-draft.md` (UofR) — show Week 4–7 proof	0.5

 Read this week:

- Pico SDK PDF §6: hardware_i2c (all: i2c_init, write_blocking, read_blocking)
- BME280 datasheet §4 (register map) — download from Bosch
- Beej's C Guide Ch 12–14
- Embedded Artistry: "A philosophy of software design for embedded systems"

 Review this week:

- Scoreboard pattern (Week 4): now add seed logging to your random TB — `$random % 256`
- FSM HDLBits §1 (Week 7): explain Moore vs Mealy to yourself in writing (3 sentences each)

Checkpoint:

- I2C driver skeleton compiles + prints WHO_AM_I result to UART (even if sensor not yet wired)
- Random TB generates 100+ transactions, scoreboard compares all, seed is logged

Push:

- `weeks/week-08/i2c-driver-plan.md` (plan + wiring diagram)
- `weeks/week-08/rtl-dv/random_stimulus_intro/` (random TB with scoreboard + seed)
- `weeks/week-08/hdlbits-notes.md`

- `professors/meeting-request-draft.md`
- `professors/us-target-profs.md` (NEW — 3 schools, 2 profs each, with lab URL + recent paper title)

Week 9 — rtl-uart-tx BEGINS (flagship repo) + deterministic TB + git workflow upgrade

Hours target: 38 hrs

Block	Task	Hours
DV	**rtl-uart-tx repo:** implement UART TX RTL (parameterized: DATA_BITS, CLKS_PER_BIT)	3.0
DV	Deterministic TB: test 0x00, 0xFF, 0xA5, 0x55, 0xAA — use scoreboard	2.5
DV	Waveform discipline: screenshot every test case — save to `docs/waveform-gallery/`	1.0
DV	HDLBits: 5+ problems from any unsolved section	1.0
DV	HDLBits notes	0.5
Git	Pro Git Ch 3 §3.4–3.6 (rebasing, remote branches) — then rebase your feature branch	1.0
Research	`paper/method-notes.md`: define exactly which bugs you will inject + how catch rate is measured	1.0
Research	Read paper #2 using 3-pass method — `reading-notes/papers/paper-02.md`	1.5
Career	Apply to 2 roles + 2 networking messages	0.5
Career	Send UofR professor email #1 (if not yet sent)	0.5

□ **Read this week:**

- Pro Git Ch 3 §3.4–3.6 (remote branches, rebasing)
- iverilog docs: "System tasks" section (`$display`, `$monitor`, `$dumpfile`, `$dumpvars`, `$finish`, `$stop`)
- IEEE 1364 (Verilog LRM) §1 + §2 (scope + overview) — 20 pages via university IEEE Xplore access

□ **Review this week:**

- iverilog flags (Week 2): run your new UART TB with `-Wall -g2012` — fix any warnings
- GTKWave §1–3 (Week 2): open UART waveform, add color to TX signal, save `.gtkw` layout file

Checkpoint:

- UART TX passes all 5 deterministic tests
- Waveform gallery has ≥5 annotated screenshots (label what each one shows)
- `rtl-uart-tx/README.md` explains how to build + run (exact commands)

Push (to `rtl-uart-tx` repo):

- `rtl/uart_tx.v` (parameterized)
- `tb/uart_tx_tb.v` (deterministic, with scoreboard)
- `docs/waveform-gallery/` (5 screenshots)
- `docs/test-plan.md` (from Week 5, now linked from README)

- `README.md` (how to build + run)

Push (to `embedded-journey`):

- `weeks/week-09/project-link.md`
- `reading-notes/papers/paper-02.md`
- Updated trackers

Week 10 — Randomized TB + scoreboard + seed logging + buy logic analyzer + RA push

Hours target: 37 hrs

Block	Task	Hours
DV	Randomized UART TB: random bytes, random idle gaps, random baud variation	2.5
DV	Scoreboard: compare expected vs actual byte-by-byte; failure includes test name + seed	2.0
DV	Seed logging: log `$random` seed to file — prove reproducibility	1.0
DV	Inject 3 deliberate bugs — confirm TB catches all 3 — document in `docs/bug-diary.md`	1.5
DV	HDLBits: 5+ problems	1.0
Hardware	Buy logic analyzer (if not yet ordered)	—
Research	RA pipeline: pick 3 UofR professor targets — write "how I can help in 2 weeks" bullet for each	1.0
Research	`professors/uofr-target-profs.md` — 3 targets with tailored notes	0.5
Research	`paper/research-log.md` entry: first randomized test observations vs deterministic	0.5
Career	Apply 3 roles + 2 networking messages	0.5

 Read this week:

- GTKWave User Guide PDF §4 (signal grouping, colors) + §5 (saving `.gtkw` layout)

 Review this week:

- UART RTL (Week 9): read your RTL again — find 1 thing you'd improve and note it
- Paper #1 summary (Week 5): re-read your summary — does it fit your research question?

Checkpoint:

- TB runs 1000+ random transactions without false failures
- Bug diary entry #1: describes bug, how it was found, root cause, fix

Push (rtl-uart-tx):

- `tb/uart_tx_rand_tb.v`
- `docs/bug-diary.md` (entry #1–#3)
- `scripts/seed_log.py` (logs seed to file)

Push (embedded-journey):

- `professors/uofr-target-profs.md`
- Updated trackers

Week 11 — Regression runner + assertions mindset + Verilator install + GitHub Actions CI

Hours target: 38 hrs

Block	Task	Hours
DV	Python regression runner: `scripts/run_tests.py` — runs all TBs, prints PASS/FAIL summary	2.5
DV	Add: test name, seed, elapsed time, failure message to summary	1.5
DV	**GitHub Actions CI:** create `.github/workflows/ci.yml` — runs regression on every push	2.5
DV	Assertions mindset: add `assert` statements to TB (invariants: START bit always low, etc.)	1.5
DV	Verilator install from source in WSL2 (follow official guide)	1.5
DV	HDLBits: 5+ problems	1.0
C	Beej's Ch 20–22 (bitwise ops, practice problems)	1.5
Research	Pro Git Ch 5 §5.1–5.2 (distributed workflows)	0.5
Career	Apply 5 roles (volume week)	0.5

Read this week:

- Pro Git Ch 5 §5.1–5.2 (distributed workflows)
- Missing Semester Lecture 6 (Git deep dive) + Lecture 9 (security awareness)
- iverilog docs: "Compiler directives" section (`` `timescale ``, `` `define ``, `` `include ``)
- Python docs: `subprocess` module (for running iverilog from Python script)

Review this week:

- Scoreboard pattern (Week 4 + 10): re-read your TB — does the failure message include seed? test name? expected vs actual? Improve if not.
- Beej Ch 6–9 (Week 7): write one program that uses structs + pointers + functions together

Checkpoint:

- One command (`python3 run_tests.py`) runs all tests and prints summary
- GitHub Actions CI badge is GREEN in `rtl-uart-tx` README
- Verilator installs without errors (even if not yet used)

Push (rtl-uart-tx):

- `scripts/run_tests.py`
- `.github/workflows/ci.yml`
- `README.md` updated with CI badge

Push (embedded-journey):

- `weeks/week-11/hdlbits-notes.md`
- `paper/research-log.md` entry: baseline vs random test catch rate (first numbers)

Week 12 — FIFO + ready/valid + Nand2Tetris Project 3 + Beej Ch 23–25

Hours target: 36 hrs

Block	Task	Hours
DV	FIFO design: synchronous FIFO (parameterized depth + width)	2.0
DV	FIFO TB: ordering tests, full/empty, overflow/underflow, backpressure	2.5
DV	Ready/valid handshake: add to UART TX (data accepted only when ready)	1.5
DV	HDLBits: 5+ problems	1.0
DV	Nand2Tetris Project 3: sequential logic (DFF, register, RAM8, RAM64)	2.0
DV	IEEE 1364 §12: behavioral modeling (blocking vs non-blocking — read in full)	1.0
C	Beej's Ch 23–25 (error handling, signals, preprocessor)	1.5
Research	Read 2 paper abstracts in target lab area — `reading-notes/abstract-01.md`, `abstract-02.md`	1.0
Research	`paper/related-work-matrix.csv` — grow to 8 entries	0.5
Career	Apply 2 roles + 2 networking messages	0.5

 Read this week:

- Beej's C Guide Ch 23–25
- IEEE Std 1364 §12: Behavioral modeling (blocking vs non-blocking — critical for DV interviews)
- YouTube: "Verilog FIFO tutorial" (watch 1 video ≥15 min)

 Review this week:

- UART RTL (Week 9): now add the ready/valid interface — does it break existing TBs? Fix them.
- HDLBits FSM §1+§2 (Weeks 7–8): explain in writing: when do you use Mealy vs Moore?

Checkpoint:

- FIFO module + TB: ordering and full/empty behavior verified
- UART TX has ready/valid interface and still passes all existing TBs

Push:

- `weeks/week-12/rtl-dv/fifo/`
- `notes/week-12-fifo-ready-valid.md`
- `rtl-uart-tx/rtl/uart_tx.v` updated (ready/valid)
- `reading-notes/abstract-01.md`, `abstract-02.md`

Week 13 — Research reading matrix + related-work v0 + RA email drafts + Schimel Ch 1–3

Hours target: 35 hrs

Block	Task	Hours
Research	Read Keshav PDF again (re-read before building matrix)	0.3

Block	Task	Hours
Research	`paper/related-work-matrix.csv` — grow to 12+ entries (title, venue, year, method, result, gap, relevance to your work)	2.0
Research	Read paper #3 using 3-pass method — `reading-notes/papers/paper-03.md`	1.5
Research	"Writing Science" by Schimel Ch 1–3: story structure of papers	1.5
Research	Draft `paper/related-work.md` v0 (2+ pages, connecting papers to your question)	2.0
DV	HDLBits: 5+ problems	1.0
DV	DV habit: add 1 new test case category to UART TB (e.g., back-to-back bytes)	1.0
Career	Draft 1 tailored RA email to UofR professor — link repo + research question + 2-week deliverable	1.0
Career	Draft 1 tailored email to US professor at target school — link repo + ask about PhD/RA	1.0
Career	Apply 2 roles	0.5

 Read this week:

- Keshav PDF (re-read, 20 min)
- "Writing Science" Ch 1–3 (buy or library — this is critical for top-10 applications)
- Google Scholar: search your research question, scan 5 more abstracts

 Review this week:

- Research question (Week 4): restate it clearly in 1 sentence. Is it still the right question?
- Paper #1 + #2 summaries: how do they relate to each other? Write 2 sentences connecting them.

Checkpoint:

- Related-work matrix: 12+ entries with gap analysis column filled
- RA email draft (UofR) is ready to send — shows repo proof + 2-week deliverable
- US professor email draft #1 exists

Push:

- `reading-notes/papers/paper-03.md`
- `paper/related-work-matrix.csv` (12+ entries)
- `paper/related-work.md` v0
- `professors/email-uofr-prof-lastname.md`
- `professors/email-us-prof-01.md`

Week 14 — Pico UART CLI + logic analyzer first use + DV coverage mindset intro

Hours target: 36 hrs

Block	Task	Hours
Hardware	Pico UART structured logs: timestamp + severity + module tag	2.0
Hardware	UART CLI: parse commands (start, stop, status, reset)	2.0
Hardware	Logic analyzer: capture UART frame, verify against your RTL spec	2.0

Block	Task	Hours
DV	Coverage mindset: add simple functional coverage counters to UART TB	1.5
DV	GTKWave: use edge search to find specific UART start bit — save `.gtkw` layout	0.5
DV	HDLBits: 5+ problems	1.0
C	Read Embedded Artistry: "Error handling in C: return codes"	0.5
Research	`paper/research-log.md` entry: logic analyzer vs simulation — what matches, what doesn't	0.5
Career	Apply 2 roles + 2 networking messages	0.5

 Read this week:

- Pico SDK PDF §7: hardware_uart (all: uart_init, uart_puts, uart_getc, uart_is_readable)
- GTKWave User Guide PDF §6: searching for signal edges
- Embedded Artistry: "Error handling in C: return codes"

 Review this week:

- iverilog system tasks (Week 9): in your CLI TB, are you using `$monitor` appropriately? Add it.
- Regression runner (Week 11): run it now — does it still pass? (Regression integrity check)

Checkpoint:

- Pico prints structured logs and responds to ≥4 CLI commands
- Logic analyzer capture shows correct UART frame matching your spec

Push:

- `weeks/week-14/pico-uart-cli-mini/` (source + logic analyzer screenshot)
- `rtl-uart-tx` updated with coverage counters
- Updated trackers

Week 15 — I2C sensor driver + datasheet mastery + DV: structured logging + send RA emails

Hours target: 36 hrs

Block	Task	Hours
Hardware	I2C: read WHO_AM_I reliably from BME280	1.0
Hardware	I2C driver: calibration data read + temperature/pressure calculation	3.0
Hardware	Stream sensor data to UART log — timestamp + value + unit	1.0
Hardware	Error: handle I2C NAK, timeout, retry with backoff	1.5
DV	UART TB: add structured log output (module, test case name, expected, actual, PASS/FAIL)	1.0
DV	HDLBits: 5+ problems	1.0
Research	Send RA email #1 to UofR professor (if not yet sent)	0.5
Research	Send US professor email #1	0.5
Career	Update `internship/target-companies.csv` with sensor/industrial companies	0.5

 Read this week:

- BME280 (or MPU-6050) datasheet §4 (register map) — full section carefully
- SparkFun Learn: "I2C" tutorial (https://learn.sparkfun.com/tutorials/i2c)
- Pico SDK PDF §6 review: i2c_write_blocking + i2c_read_blocking return values and error codes

 Review this week:

- I2C driver skeleton (Week 8): your plan vs your implementation — what changed?
- Regression runner (Week 11): add your new UART structured log TB to the test suite

Checkpoint:

- Sensor streams clean readings for 10 minutes with correct values (compare to room temp)
- All I2C functions return error codes — no void functions in the driver

Push:

- `weeks/week-15/i2c-sensor-driver/` (source + 10-min sensor log file)
- `weeks/week-15/datasheet-summary.md`
- `professors/outreach-tracker.csv` updated

Week 16 — pico-data-logger repo starts + spec + architecture + MISRA-aware C + RA meeting prep

Hours target: 36 hrs

Block	Task	Hours
Hardware	pico-data-logger: define architecture (sensor → ring buffer → UART log → (future) SD)	1.5
Hardware	Write full `docs/spec.md`: interfaces, sampling rate, buffer size, error states	1.5
Hardware	Scaffold project: `src/`, `include/`, `docs/`, `Makefile`	1.0
Hardware	Implement ring buffer (from your Week 7 C knowledge) + unit test in C	2.0
C	MISRA-aware habits: run `cppcheck` on all your C files so far — fix warnings	1.5
DV	HDLBits: 5+ problems	1.0
DV	DV habit: add 1 new negative test to UART regression	0.5
Research	`professors/meeting-packet-outline.md` (RA meeting prep)	0.5
Research	Read Schimel Ch 4–6: OCAR framework — apply to outline your paper	1.0
Career	Apply 2 roles	0.5

 Read this week:

- Pico SDK PDF §8: hardware_spi (skim — for future SD card)
- "Writing Science" Ch 4–6 (Schimel)
- cppcheck documentation: how to run, what warnings mean (https://cppcheck.sourceforge.io/)

 Review this week:

- Ring buffer (Embedded Artistry, Week 7): re-read the article, then compare to your implementation
- Beej Ch 12–14 (Week 8): run valgrind on your C programs — fix any memory leaks

Checkpoint:

- Ring buffer implementation passes unit tests (overflow, underflow, wrap-around)
- cppcheck runs clean (0 warnings) on all your C files

Push:

- `pico-data-logger/` repo: README + `docs/spec.md` + scaffold + ring buffer
- `professors/meeting-packet-outline.md`
- `weeks/week-16/project-plan.md`

Week 17 — Data logger robustness: fault FSM + retry/backoff + DV: coverage reporting

Hours target: 36 hrs

Block	Task	Hours
Hardware	Fault state machine: INIT → RUNNING → FAULT → RECOVERY → RUNNING	2.5
Hardware	Retry with exponential backoff on I2C errors	1.5
Hardware	Logger runs 30+ minutes — log every fault event to UART with timestamp	1.0
Hardware	Parse fault log manually: how many faults? How long did recovery take?	0.5
C	Implement backoff in C using your non-blocking timer	1.0
DV	`scripts/coverage_report.py`: count distinct byte values exercised in UART TB	1.5
DV	HDLBits: 5+ problems + notes	1.0
Research	`paper/experiment-01-plan.md`: define baseline experiment (deterministic-only, catch rate)	1.0
Career	Apply 5 roles (volume week)	0.5

 Read this week:

- Embedded Artistry: "Timeouts and retries in embedded systems"
- Pico datasheet §4: GPIO electrical specs + UART pin mux (download from raspberrypi.com)
- Beej Ch 23–25 review: re-read error handling chapter

 Review this week:

- Fault FSM: draw the state diagram from memory — then check against your code
- Coverage report: re-run your UART regression; does coverage_report.py show gaps?
- cppcheck (Week 16): run again on all new Week 17 C files

Checkpoint:

- Logger runs 30 min with structured fault log
- `coverage_report.py` runs and shows: # distinct bytes tested / 256 (e.g., 47/256 = 18%)

- `experiment-01-plan.md` exists with exact measurement protocol written

Push:

- `pico-data-logger/`: robustness changes + 30-min log file + fault event table
- `rtl-uart-tx/scripts/coverage_report.py`
- `paper/experiment-01-plan.md`
- Updated trackers

Week 18 — Data logger CLI + Python parser + DV: experiment baseline runs

Hours target: 36 hrs

Block	Task	Hours
Hardware	UART CLI: `start`, `stop`, `rate <hz>`, `status`, `reset`, `dump` commands	2.0
Hardware	Config: save sampling rate to flash (Pico flash write using SDK)	1.5
Python	`tools/log_parser.py`: parse UART log → CSV → print stats (min, max, mean, stddev)	2.5
Python	Plot sensor data with matplotlib: time-series graph saved as PNG	1.0
DV	Run **experiment #1 (baseline)**: deterministic UART tests only	1.5
DV	Record: # tests, # bugs injected, # caught, catch rate % → `paper/experiment-01-results.md`	1.0
DV	HDLBits: 5+ problems + notes	1.0
Research	Draft `professors/email-uofr-prof-final.md` (final polished RA email)	0.5
Career	Apply 2 roles + 2 networking messages	0.5

Read this week:

- Python docs: `csv` module + `statistics` module (https://docs.python.org/3/library/csv.html)
- Python docs: `argparse` module (for making `log_parser.py` a proper CLI tool)
- Matplotlib quickstart: https://matplotlib.org/stable/tutorials/introductory/quick_start.html

Review this week:

- Ring buffer (Week 16): does your logger ever drop samples? Add a drop counter.
- UART CLI (this week): test all 6 commands — does reset restore to clean state?

Checkpoint:

- `dump` command outputs full buffer as CSV over UART
- `log_parser.py` produces CSV + plot PNG in one command
- Experiment #1 results table exists with honest numbers

Push:

- `pico-data-logger/`: CLI + flash config + `tools/log_parser.py` + example plot PNG + `DEMO.md`
- `paper/experiment-01-results.md`
- `professors/email-uofr-prof-final.md`

Week 19 — DV polish: coverage mindset deep + bug diary discipline + write-up of DV method

Hours target: 35 hrs

Block	Task	Hours
DV	Coverage mindset: add coverage bins to UART TB - Bin 1: all-zeros byte - Bin 2: all-ones byte - Bin 3: alternating bit pattern - Bin 4: random bytes - Bin 5: back-to-back bytes (no idle gap)	1.5
DV	Test reduction: take a failing random test, reduce seed to minimal reproduction	1.5
DV	Bug diary: write entry #4–#5 (root cause analysis for each)	1.0
DV	HDLBits: 5+ problems + notes	1.0
Research	Write `paper/method.md` v0: describe your DV methodology (2–3 pages)	2.5
Research	`paper/research-log.md`: entry comparing baseline vs upcoming random experiment	0.5
Career	Message 2 DV engineers on LinkedIn: "What do you test day-to-day at your company?"	0.5
Career	Apply 2 roles	0.5

 Read this week:

- Embedded Artistry: "Writing defensive code"
- HDLBits wiki: re-read the explanation pages for any section you've completed
- Schimel Ch 4–6 review: re-read OCAR framework before writing `method.md`

 Review this week:

- Experiment #1 results (Week 18): are the numbers honest? Would a professor believe them?
- Bug diary entries #1–#3 (Weeks 10, 14): are root causes explained clearly enough for a paper?

Checkpoint:

- Coverage bins all hit at least once in your regression
- Test reduction documented: from N transactions to minimal failing case
- `paper/method.md` v0 exists (even rough — 2+ pages describing what you built and why)

Push:

- `rtl-uart-tx/tb/` updated with coverage bins
- `rtl-uart-tx/docs/bug-diary.md` (entries #1–#5)
- `paper/method.md` v0
- Updated networking tracker

Week 20 — RA outreach week + run experiment #2 (randomized) + US professor emails

Hours target: 35 hrs

Block	Task	Hours
DV	Run **experiment #2**: randomized UART tests (1000+ transactions, 5 seeds)	2.0
DV	Record: catch rate % vs experiment #1 → `paper/experiment-02-results.md`	1.0
DV	Run regression: both deterministic + randomized — compare summary tables	0.5
DV	HDLBits: 5+ problems + notes	1.0
Research	Send UofR RA emails: **2–3 tailored emails**, set follow-up dates	1.0
Research	Send US professor emails: **2–3 tailored emails** to target PhD/MS supervisors	1.0
Research	Read paper #4 using 3-pass method — `reading-notes/papers/paper-04.md`	1.5
Research	`professors/meeting-prep.md`: agenda for any RA meeting that comes back	0.5
Career	Apply 2 roles	0.5
Career	Follow up on any unanswered Week 6 + 15 professor emails	0.5

Read this week:

- Columbia career education: "Emailing professors" https://www.careereducation.columbia.edu/resources/emailing-professors
- Keshav PDF (re-read once before paper #4)

Review this week:

- Experiment #1 vs #2: write a 1-paragraph plain-English summary of what the numbers show
- Coverage report (Week 17): re-run with experiment #2 data — does randomization cover more bins?

Checkpoint:

- Both experiments run + results tables exist
- 2–3 UofR RA emails sent + follow-ups scheduled
- 2–3 US professor emails sent

Push:

- `paper/experiment-02-results.md`
- `professors/outreach-tracker.csv` updated (UofR + US professors)
- `professors/meeting-prep.md`
- Updated trackers

Week 21 — Research experiments: third experiment + first result figure + Python plotting

Hours target: 35 hrs

Block	Task	Hours

Block	Task	Hours
DV	Run **experiment #3**: inject 10 different bug types, compare deterministic vs random catch rate	2.5
DV	Record: per-bug-type catch rate → `paper/experiment-03-results.md`	1.0
Python	`paper/scripts/plot_results.py`: generate bar chart comparing experiments	1.5
Python	First result figure: bug catch rate % (deterministic vs random) — save as PNG	0.5
DV	HDLBits: 5+ problems + notes	1.0
Research	Follow up on any professor emails (7-day follow-up rule)	0.5
Research	`paper/research-log.md` entry: describe first result figure	0.3
Career	Apply 2 roles + 2 networking messages	0.5

Read this week:

- Matplotlib: bar charts + figure saving https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.savefig.html
- Python docs: `pathlib` module (clean file path handling)
- YouTube: "matplotlib tutorial for engineers" — watch 1 video (20 min)

Review this week:

- All 3 experiments: are your bug types consistent across experiments? If not, re-run with consistent methodology.
- `paper/method.md` v0: update it to reflect all 3 experiments

Checkpoint:

- Experiment #3 complete — per-bug-type data exists
- Result figure #1 exists as PNG in `paper/figures/`
- Python plotting script runs in one command

Push:

- `paper/experiment-03-results.md`
- `paper/figures/fig01-catch-rate.png`
- `paper/scripts/plot_results.py`

Week 22 — Paper: outline + contributions + intro v0 + advanced DV: CDC concept

Hours target: 36 hrs

Block	Task	Hours
Research	`paper/outline.md`: full paper structure (Abstract, Intro, Related Work, Method, Results, Discussion, Conclusion)	1.0
Research	`paper/contributions.md`: 3–4 bullet point contributions (be specific, not vague)	0.5
Research	`paper/intro.md` v0: write 1.5–2 pages (motivation, gap, your approach, contributions)	2.5
Research	Schimel Ch 7–9: writing sentences that work — apply to intro revision	1.0
DV	CDC concept: watch 2 videos "clock domain crossing basics" — take notes	1.0

Block	Task	Hours
DV	Write `notes/week-22-cdc-concept.md`: what is CDC, why it causes failures, what a synchronizer does	1.0
DV	HDLBits: 5+ problems + notes	1.0
Career	Follow up on RA emails; request short meeting	0.5
Career	Apply 2 roles	0.5

 Read this week:

- Schimel "Writing Science" Ch 7–9
- YouTube: "clock domain crossing CDC basics" (watch 2 videos, take bullet notes)
- IEEE 1364 §14: Specify blocks + timing checks (skim — awareness level)

 Review this week:

- `paper/related-work.md` v0 (Week 13): revise based on your new experiments
- Contributions bullet points: are they specific and falsifiable? (e.g., "Random testing catches 40% more bugs than deterministic in our UART DUT" — not "we improved testing")

Checkpoint:

- Paper outline exists with all sections
- `intro.md` v0: 1.5+ pages, clear motivation + gap + your approach
- CDC concept notes written — you can explain it in 5 minutes

Push:

- `paper/outline.md`
- `paper/contributions.md`
- `paper/intro.md` v0
- `notes/week-22-cdc-concept.md`

Week 23 — Advanced IC: CDC + reset + metastability + timing (interview-critical + paper depth)

Hours target: 36 hrs

Block	Task	Hours
DV	Implement 2-FF synchronizer in Verilog: `rtl/cdc_sync2.v`	1.5
DV	Write TB for synchronizer: test with asynchronous input, verify stability after sync	2.0
DV	Reset strategy: implement synchronized reset in UART TX — update existing RTL	1.5
DV	Write `notes/week-23-reset-strategy.md`: sync reset vs async reset vs async assert / sync deassert	1.0
DV	Setup/hold: watch 2 videos "setup hold time explained" — write `notes/week-23-timing.md`	1.0
DV	HDLBits: 5+ problems + notes	1.0
Research	`paper/research-log.md` entry: does CDC apply to your UART DUT? (even if not, explain why)	0.3

Block	Task	Hours
Career	Apply 5 roles (volume week)	0.5

 Read this week:

- iverilog docs: "Known issues / limitations" page (for debugging awareness)
- IEEE 1364 §14: Specify blocks + timing checks (finish reading)
- YouTube: "setup hold time explained" (2 videos)
- YouTube: "clock domain crossing CDC" (1 more advanced video)

 Review this week:

- UART TX RTL (Week 9 + Week 12): now with synchronized reset — does regression still pass?
- GTKWave §6 (Week 14): use edge search to find your synchronizer output stabilizing

Checkpoint:

- 2-FF synchronizer + TB passes
- UART TX has async-assert sync-deassert reset and passes full regression
- You can explain: setup time, hold time, metastability, synchronizer in one 5-minute explanation

Push:

- `rtl-uart-tx/rtl/cdc_sync2.v` + TB
- `rtl-uart-tx/rtl/uart_tx.v` updated (sync reset)
- `notes/week-23-cdc-reset.md`
- `notes/week-23-timing.md`
- CI badge re-verified (regression still green)

Week 24 — Portfolio packaging: recruiter + professor + MS admissions quality polish

Hours target: 34 hrs

Block	Task	Hours
Portfolio	`embedded-journey/README.md`: full professional landing page rewrite	2.0
Portfolio	`rtl-uart-tx/README.md`: add results table, waveform gallery section, CI badge, exact commands	2.0
Portfolio	`pico-data-logger/README.md`: add architecture diagram, sample plot, exact commands	1.5
Portfolio	Both repos: verify reproducibility on fresh WSL2 terminal (no aliases, no shortcuts)	1.5
Portfolio	Add `CONTRIBUTING.md` to both project repos (professional signal)	0.5
Portfolio	Pro Git Ch 8 §8.1–8.3: Git hooks — add a pre-commit hook that runs cppcheck	1.0
DV	HDLBits: 5+ problems + notes	1.0
Research	Read paper #5 — `reading-notes/papers/paper-05.md`	1.5
Career	Ask 2 people to review your `embedded-journey` README	0.3
Career	Apply 2 roles	0.5

 Read this week:

- Pro Git Ch 8 §8.1–8.3 (Git hooks)
- makeareadme.com: full guide (https://www.makeareadme.com/)
- GitHub Actions docs: badges section https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/adding-a-workflow-status-badge

 Review this week:

- All CI: trigger a fresh push — both project repos should have green badges
- Bug diary (Weeks 10–19): are entries #1–#5 written clearly enough for a professor to read?

Checkpoint:

- `embedded-journey/README.md` reads as a complete professional portfolio
- Both project repos: green CI badge + results table + waveform gallery + reproducible commands
- Pre-commit hook catches cppcheck errors before every commit

Push:

- All three README s updated
- `pico-data-logger/assets/` and `rtl-uart-tx/docs/waveform-gallery/` fully populated
- `CONTRIBUTING.md` in both project repos

Week 25 — Interview prep: DV + embedded + 30 Q&As + mock interview

Hours target: 35 hrs

Block	Task	Hours
Interview	Write `weeks/week-25/interview-qa-dv.md`: 20 DV questions + your written answers	3.0
Interview	Topics to cover in DV Q&A: - Explain your testbench architecture - What is a scoreboard and why use one? - What is a seed and why log it? - What is functional coverage? How do you measure it? - What is CDC? What is a synchronizer? - Blocking vs non-blocking assignment — when do you use each? - What is metastability? - How do you debug a failing simulation? Walk me through your steps. - What does your regression runner do? - Describe a bug you found. Root cause? Fix?	
Interview	Write `weeks/week-25/interview-qa-embedded.md`: 20 embedded questions + answers	2.5
Interview	Topics for embedded Q&A: - `volatile` keyword — when and why? - Interrupt vs polling — tradeoffs - What is a ring buffer?	

Block	Task	Hours
	- Explain I2C transaction: start, address, ACK, data, stop - What is a non-blocking scheduler? - What does `cppcheck` catch that the compiler doesn't? - What is exponential backoff? - Explain UART frame: start bit, data bits, stop bit - What is a fault state machine? - What causes stack overflow on an MCU?	
Interview	Record 1 mock interview (camera on, timer on, 30 min) — watch back and critique	1.5
DV	HDLBits: 5+ problems + notes	1.0
Career	Apply 2 roles + 2 networking messages	0.5

 Read this week:

- YouTube: "Design verification interview questions" (1 video ≥20 min)
- YouTube: "Embedded C interview questions" (1 video ≥20 min)

 Review this week:

- CDC + reset (Week 23): can you explain it without notes in 5 minutes? Record yourself.
- All HDLBits notes (Weeks 2–24): flip through and re-solve 5 problems you got wrong

Checkpoint:

- 40 Q&As written (20 DV + 20 embedded)
- Mock interview recorded
- You can answer all 20 DV questions without notes

Push:

- `weeks/week-25/interview-qa-dv.md`
- `weeks/week-25/interview-qa-embedded.md`
- `weeks/week-25/mock-interview-notes.md` (what you noticed watching yourself)

Week 26 — Paper: intro v1 + related work v1 + citations discipline

Hours target: 35 hrs

Block	Task	Hours
Research	`paper/intro.md` v1: revise based on Schimel Ch 7–9 — tighten every sentence	2.0
Research	`paper/related-work.md` v1: full section (3–4 pages), 10+ cited papers	3.0
Research	Zotero: organize all papers — generate reference list in IEEE format	1.0
Research	Send 1 section (`intro.md` v1) to UofR professor or mentor for feedback	0.3
Research	Read paper #6 — `reading-notes/papers/paper-06.md`	1.5
DV	HDLBits: 5+ problems + notes	1.0
Career	Apply 2 roles + 2 networking messages	0.5

 Read this week:

- Zotero quick start: https://www.zotero.org/support/quick_start_guide
- IEEE reference format guide (search "IEEE citation format PDF")
- Schimel Ch 7–9 (re-read key sections on sentence construction)

 Review this week:

- `paper/contributions.md` (Week 22): do your intro and related work support these contributions?
- All paper summaries (#1–#5): can you state the gap each paper leaves in 1 sentence?

Checkpoint:

- Related work section: 10+ papers cited, each with a gap analysis sentence
- Intro v1 revised — shorter, sharper, clearer
- Zotero library has all 10+ papers with IEEE citations

Push:

- `paper/intro.md` v1
- `paper/related-work.md` v1
- `paper/references.bib` (or `references.md` in IEEE format)
- `reading-notes/papers/paper-06.md`

Week 27 — Paper: method v1 + experiments section v1 + RA meeting prep + GRE awareness

Hours target: 35 hrs

Block	Task	Hours
Research	`paper/method.md` v1: full methodology (3+ pages, reproducible, exact tool versions)	3.0
Research	`paper/experiments.md` v1: all 3 experiments described with exact protocol	2.0
Research	`professors/2-week-deliverables.md`: what you will produce for RA meeting	0.5
Research	RA meeting: prepare agenda — bring laptop, repos open, results table printed	0.5
DV	HDLBits: 5+ problems + notes	1.0
Career	GRE: research which target schools require it — log in `notes/ms-application-tracker.md`	0.5
Career	Start `notes/ms-application-tracker.md`: school, program, deadline, GRE needed, funding type	0.5
Career	Apply 2 roles	0.5

 Read this week:

- YouTube: "How to write methodology section research" (1 video)
- Read your own `paper/experiment-01/02/03-results.md` critically: are the numbers presented consistently?

 Review this week:

- Experiment results (Weeks 18–21): unify all tables into one consistent format with same column names
- `paper/method.md` v0 (Week 19): upgrade to v1 now — add exact tool versions, exact commands, exact bug list

Checkpoint:

- `method.md` v1: any independent researcher could repeat your experiments from this document
- `experiments.md` v1: all 3 experiments described, results referenced correctly
- `ms-application-tracker.md` seeded with 6–8 target schools

Push:

- `paper/method.md` v1
- `paper/experiments.md` v1
- `professors/2-week-deliverables.md`
- `notes/ms-application-tracker.md`

Week 28 — Paper: results + discussion v1 + second result figure + RA delivery

Hours target: 36 hrs

Block	Task	Hours
Research	`paper/results.md` v1: present all quantitative results with figures (2–3 figures)	2.5
Research	`paper/discussion.md` v1: what do results mean? limitations? threats to validity?	2.0
Research	Generate figure #2: per-bug-type catch rate table as bar chart	1.0
Research	Generate figure #3: coverage growth vs number of random test transactions	1.0
Research	Deliver RA task to professor (whatever was agreed in Week 27 meeting)	1.5
DV	HDLBits: 5+ problems + notes	1.0
Career	Follow up with RA targets showing updated results	0.5
Career	Apply 2 roles	0.5

 Read this week:

- YouTube: "How to write discussion section research paper" (1 video)
- Schimel: re-read the chapter on "what story your data tells"

 Review this week:

- All 3 experiment results: are all numbers consistent across `method.md`, `experiments.md`, `results.md`?
- Figures: do they have axis labels, title, legend, caption? Check each one.

Checkpoint:

- Results section: 3 figures with captions, quantitative data, honest limitations
- Discussion: addresses "why randomized testing catches more bugs" with evidence from your data

- RA deliverable submitted to professor

Push:

- `paper/results.md` v1
- `paper/discussion.md` v1
- `paper/figures/fig02-per-bug-catch-rate.png`
- `paper/figures/fig03-coverage-growth.png`
- RA deliverable committed to appropriate location

Week 29 — Paper: abstract + full draft assembly + conclusion + submit target identified

Hours target: 36 hrs

Block	Task	Hours
Research	`paper/abstract.md`: write abstract last (250 words max)	1.5
Research	Abstract structure: background (1 sentence) → gap (1 sentence) → your approach (2 sentences) → key results (2 sentences) → conclusion (1 sentence)	—
Research	`paper/conclusion.md`: write conclusion (recap contributions + future work)	1.0
Research	`paper/full-draft.md`: assemble all sections into one document	1.5
Research	Identify 1 target submission venue: - **IEEE SSCS Student Paper Contest** (if eligible) - **DAC Student Research Competition (SRC)** - **Local/regional undergraduate research symposium** (minimum baseline) - **IEEE Region 7 (Canada) student paper competition**	1.0
Research	Read submission guidelines of your target venue — note word limit + deadline	0.5
DV	HDLBits: 5+ problems + notes	1.0
Career	Ask UofR professor to co-author or review full draft (critical for credibility)	0.5
Career	Apply 2 roles	0.5

 Read this week:

- YouTube: "How to write an abstract for a research paper" (1 video)
- IEEE SSCS Student Paper Contest page (search current year)
- DAC Student Research Competition page (dac.com/attend/student-research-competition)

 Review this week:

- `paper/contributions.md` (Week 22): does your abstract match your contributions exactly?
- All figures: view them at conference-paper zoom level (small) — are they still readable?

Checkpoint:

- Full draft assembled: Abstract + Intro + Related Work + Method + Experiments + Results + Discussion + Conclusion
- Target submission venue identified + deadline noted
- Professor asked for feedback or co-authorship

Push:

- `paper/abstract.md`
- `paper/conclusion.md`
- `paper/full-draft.md` (or PDF if using LaTeX)
- `paper/submission-target.md` (venue + deadline + guidelines summary)

Week 30 — DV advanced: regression dashboard + trend tracking + paper peer review response

Hours target: 36 hrs

Block	Task	Hours
DV	Regression dashboard: upgrade `run_tests.py` to write `regression_history.json`	2.0
DV	History file: stores per-run: date, # tests, # passed, # failed, seeds used	1.0
DV	`scripts/trend_plot.py`: plot pass rate over time (regression health chart)	1.5
DV	Add regression trend plot to `rtl-uart-tx/README.md`	0.5
DV	HDLBits: 5+ problems + notes	1.0
Research	Incorporate professor feedback into `paper/full-draft.md` v2	2.0
Research	`paper/research-log.md` entry: reflection on full draft — what is weakest section?	0.3
Career	Apply 5 roles (volume week)	0.5
Career	Update `notes/ms-application-tracker.md`: deadlines getting closer — mark urgent	0.5

 Read this week:

- Python docs: `json` module (for history file)
- Python docs: `datetime` module (for timestamps in history)
- Matplotlib: line charts with date axis

 Review this week:

- Full paper draft: read it aloud — where do you stumble? Those are the sentences to fix.
- UART TX full regression: run clean with no modifications — CI should be green

Checkpoint:

- Regression dashboard: single command shows today's run + trend over all weeks
- `paper/full-draft.md` v2 incorporates professor feedback

Push:

- `rtl-uart-tx/scripts/run_tests.py` upgraded
- `rtl-uart-tx/scripts/trend_plot.py`
- `rtl-uart-tx/docs/regression-trend.png`
- `paper/full-draft.md` v2

Week 31 — Capstone polish: refactor + architecture diagrams + SOP draft begins

Hours target: 36 hrs

Block	Task	Hours
Portfolio	Refactor `rtl-uart-tx` RTL: clean up naming, add header comments, parameter documentation	1.5
Portfolio	Refactor `pico-data-logger`: consistent error code enum, all functions documented	1.5
Portfolio	Architecture diagrams: draw for both projects (use draw.io or Excalidraw — free)	2.0
Portfolio	Add diagrams to both repos in `docs/`	0.5
Portfolio	`embedded-journey/README.md` final: add architecture diagram thumbnails + paper link	1.0
SOP	**Statement of Purpose draft begins**	2.0
	`notes/sop-draft-v0.md` structure:	
	§1: Why IC design / DV? (your story — 1 paragraph)	
	§2: What you built (UART DV project — concrete, with numbers)	
	§3: Your research (the paper — 1 paragraph on methodology + result)	
	§4: Why this school / professor (tailored per application)	
	§5: Career goal (where you want to be in 5 years)	
DV	HDLBits: 5+ problems + notes	1.0
Career	Ask 2 people to review portfolio README	0.3

 Read this week:

- Excalidraw guide: https://excalidraw.com/ (try it — free, browser-based diagrams)
- MIT/Stanford SOP examples (search "successful EECS SOP examples" — read 2–3)
- makeareadme.com: re-read the guide with fresh eyes

 Review this week:

- Interview Q&As (Week 25): re-read all 40 — add any new answers you've learned since Week 25
- MS application tracker (Week 27): update all deadlines, confirm GRE requirement status

Checkpoint:

- Architecture diagrams exist for both projects and are linked in READMEs
- SOP draft v0 exists (even rough — all 5 sections have at least 1 paragraph each)
- Portfolio README reviewed by at least 1 external person

Push:

- Both repos: architecture diagrams added to `docs/`
- READMEs updated with diagrams
- `notes/sop-draft-v0.md`

Week 32 — RA conversion + SOP v1 + resume v2 + MS application materials

Hours target: 36 hrs

Block	Task	Hours
RA	Attend RA meeting(s): present your work, agree on next task	1.5
RA	`professors/meeting-notes-YYYY-MM-DD.md`: detailed notes + agreed deliverables	0.5
SOP	`notes/sop-draft-v1.md`: full 1000-word SOP (generic template)	2.5
SOP	Make 2 tailored versions for 2 target schools (different §4 — professor + school specifics)	1.5
Resume	`resume/resume-v2.md` or LaTeX: add all projects, DV skills, paper (under review / draft), RA position	2.0
Resume	Skills section: Verilog, SystemVerilog awareness, iverilog, GTKWave, Verilator, C (embedded), Python, Git, GitHub Actions, I2C, UART, Linux/WSL2	—
DV	HDLBits: 5+ problems + notes	1.0
Career	Apply 2 roles + 2 networking messages	0.5

 Read this week:

- MIT admissions: SOP tips (search "MIT EECS graduate application statement of purpose")
- Georgia Tech ECE MS admission: what they look for (search "gatech ece ms admission requirements")
- IEEE resume template (search "IEEE resume template LaTeX")

 Review this week:

- Paper full draft v2 (Week 30): read abstract only — does it accurately reflect what you did?
- SOP v0: is §2 (your project) specific enough? Does it mention: DV methodology, catch rate numbers, CI pipeline?

Checkpoint:

- SOP v1: 1000 words, all 5 sections complete
- 2 tailored versions exist for 2 different schools
- Resume v2 includes paper + RA position + CI badge mention

Push:

- `notes/sop-draft-v1.md`
- `notes/sop-school-A.md` (tailored)
- `notes/sop-school-B.md` (tailored)
- `resume/resume-v2.pdf`
- `professors/meeting-notes-YYYY-MM-DD.md`

Week 33 — Execute RA task + paper submission prep + SystemVerilog awareness begins

Hours target: 37 hrs

Block	Task	Hours

Block	Task	Hours
RA	Execute Week 32 RA task: produce deliverable (code / data / summary)	3.0
RA	Document deliverable clearly — format it for professor review	1.0
Research	Paper: final revision targeting submission venue format	2.0
Research	Format paper to venue template (IEEE 2-column if needed) — use LaTeX or Word	1.5
Research	`paper/submission-checklist.md`: author names, affiliations, abstract word count, figure count, references	0.5
DV	SystemVerilog awareness: watch 1 intro video "SystemVerilog for verification"	1.0
DV	Write `notes/week-33-sv-awareness.md`: what SV adds over Verilog for DV (interfaces, clocking blocks, randomization)	1.0
DV	HDLBits: 5+ problems + notes	1.0
Career	Apply 2 roles + 2 networking messages	0.5

☐ **Read this week:**

- SystemVerilog intro: "SystemVerilog for Verification" (Spear + Tumbush) — read Ch 1 only (get from library or find open access)
- Your target venue's author guidelines (full — read every word)
- IEEE LaTeX template guide: https://www.ieee.org/conferences/publishing/templates.html

☐ **Review this week:**

- All paper sections: check every figure has a number, caption, and is referenced in the text
- SOP v1: give it to 1 trusted person to read — collect feedback before Week 34

Checkpoint:

- RA deliverable produced and submitted to professor
- Paper formatted to venue template (even if not submitted yet — draft-ready)
- SV awareness notes written

Push:

- RA deliverable (in appropriate location or private notes)
- `paper/full-draft-v3.pdf` (venue-formatted)
- `paper/submission-checklist.md`
- `notes/week-33-sv-awareness.md`

Week 34 — Final integration: retrospective + next plan + paper submit + MS application push

Hours target: 36 hrs

Block	Task	Hours
Retrospective	`weeks/week-34/final-retrospective.md`: honest 34-week review What worked? What didn't? What would you do differently in Week 1? Quantitative summary: # commits, # TBs written, # bugs caught, # papers read, # professors contacted, # applications sent	2.0

Block	Task	Hours
Paper	Submit paper to target venue (or upload to arXiv as preprint if venue not ready)	1.0
Paper	`paper/full-draft.md` final version committed	0.5
MS Apps	Final SOP revisions incorporating Week 33 feedback	1.5
MS Apps	`notes/ms-application-tracker.md`: mark all schools with final deadline, materials checklist	1.0
Resume	`resume/resume-v2.pdf` final — confirmed accurate and up to date	0.5
Portfolio	`embedded-journey/README.md` absolutely final — this is what professors see	1.0
Plan	`notes/next-6-month-plan.md`: next targets after Week 34 - GRE prep (if required) - SystemVerilog deep dive - UVM fundamentals - Second paper or conference poster - Deepen RA or convert to NSERC USRA	1.5
DV	HDLBits: 5+ problems + notes (never stop)	1.0
Career	Send thank-you notes to all professors who helped	0.5

 Read this week:

- Missing Semester: re-watch Lecture 1 (you will notice things you missed the first time)
- arXiv submission guide: https://arxiv.org/help/submit (skim — know how to upload a preprint)

 Review this week:

- Everything: open your Week 1 `weekly-review.md` — compare to where you are now
- All HDLBits notes: count how many problems you solved across 34 weeks

Checkpoint — You have:

- Professional GitHub portfolio (`embedded-journey` + 2 polished project repos)
- Flagship DV repo (`rtl-uart-tx`) with CI, scoreboard, random TB, regression dashboard, waveform gallery
- Embedded project (`pico-data-logger`) with fault FSM, CLI, Python parser, logic analyzer proof
- Research paper (full draft, venue-formatted, submitted or preprint)
- RA position at UofR (active or pending)
- Contact with 2–3 US professors at target schools
- SOP ready for 3+ schools
- Resume v2 ready
- 40+ DV + Embedded interview Q&As written
- MS application tracker with deadlines

Final Push:

- `weeks/week-34/final-retrospective.md`
- `paper/full-draft-final.md` (or `.pdf`)
- `resume/resume-v2.pdf` final
- `notes/ms-application-tracker.md` complete

- `notes/next-6-month-plan.md`
- `embedded-journey/README.md` final

Post-Week-34: Next 6 Months (brief roadmap)

Month	Priority
Month 1–2	SystemVerilog deep: interfaces, clocking blocks, classes, randomization
Month 2–3	UVM fundamentals: uvm_component, uvm_object, uvm_driver, uvm_monitor, uvm_scoreboard
Month 3–4	Second project: AXI-Lite slave RTL + UVM testbench
Month 4–5	GRE prep (if required): target 167+ Quant, 158+ Verbal
Month 5–6	MS applications: submit to 6–10 schools
Ongoing	Continue RA, publish or present paper, grow professor network

---

□ MASTER SHOPPING SHEET — Amazon Canada

Everything You Need for 34 Weeks (Regina → US MS Ready)

> **Buying strategy:** Buy phases in order. Do NOT buy Phase 3+ early. All prices are approximate CAD estimates (2025–2026). Search links provided — choose seller with Prime + 4★+ rating.

PHASE 0 — Software Only (Week 1, Cost: $0)

Everything in Phase 0 is free. Install in order.

Item	Where	Cost
Git for Windows	https://git-scm.com/download/win	FREE
VS Code	https://code.visualstudio.com/	FREE
Python 3.11+	https://www.python.org/downloads/windows/	FREE
GitHub CLI	https://cli.github.com/	FREE
WSL2 + Ubuntu	Windows Store / Microsoft docs	FREE
iverilog (WSL2)	`sudo apt install iverilog`	FREE
GTKWave (WSL2)	`sudo apt install gtkwave`	FREE
Zotero	https://www.zotero.org/	FREE
Excalidraw	https://excalidraw.com/	FREE

Item	Where	Cost
Verilator	Build from source (Week 11)	FREE

PHASE 1 — Core Embedded Hardware (Buy in Week 4–5, before Week 6)

Must buy before Week 6 starts.

#	Item	Amazon.ca Search	Est. Price (CAD)	Priority
1	**Raspberry Pi Pico** (non-W is fine to start)	https://www.amazon.ca/s?k=raspberry+pi+pico	$8–$12	☐ MUST
2	**Micro-USB cable (data + charge)** — make sure it is NOT charge-only	https://www.amazon.ca/s?k=micro+usb+data+cable	$8–$12	☐ MUST
3	**Breadboard (830-tie points)**	https://www.amazon.ca/s?k=830+breadboard	$8–$12	☐ MUST
4	**Jumper wire kit (M-M + M-F + F-F)**	https://www.amazon.ca/s?k=jumper+wire+kit+male+female	$10–$15	☐ MUST
5	**2.54mm pin headers (40-pin straight)** — for soldering onto Pico	https://www.amazon.ca/s?k=2.54mm+pin+header+40+pin	$5–$8	☐ MUST
6	**BME280 breakout board (I2C/SPI)** — Bosch temp/pressure/humidity	https://www.amazon.ca/s?k=bme280+breakout	$10–$18	☐ MUST

Phase 1 Total: ~$49–$77 CAD

> ⚠ If BME280 is unavailable or expensive, acceptable alternatives:
>
> - MPU-6050 (IMU): https://www.amazon.ca/s?k=mpu-6050+module (~$8–$12)
> - DHT22 (simpler, 1-wire style): https://www.amazon.ca/s?k=dht22+sensor+module (~$8–$12)

PHASE 2 — Debugging Tools (Buy in Week 9–10)

Buy after your DV toolchain is stable and hardware is running.

#	Item	Amazon.ca Search	Est. Price (CAD)	Priority
7	**USB Logic Analyzer — 8-channel, 24MHz** (Saleae clone style, works with PulseView/sigrok)	https://www.amazon.ca/s?k=usb+logic+analyzer+24mhz+8+channel	$15–$25	☐ MUST (Week 14 needs it)
8	**Digital multimeter** (auto-ranging, basic)	https://www.amazon.ca/s?k=digital+multimeter+auto+range	$18–$35	☐ HIGH

Phase 2 Total: ~$33–$60 CAD

Software for logic analyzer (free):

- PulseView / sigrok: https://sigrok.org/wiki/PulseView
- Saleae Logic (if using genuine Saleae): https://www.saleae.com/downloads/

PHASE 3 — Soldering Tools (Buy in Week 13–14)

Buy when you are ready to permanently solder pin headers onto your Pico.

#	Item	Amazon.ca Search	Est. Price (CAD)	Priority
9	**Soldering iron station** (temperature-controlled, 60W) — NOT a cheap fixed-temp iron	https://www.amazon.ca/s?k=soldering+iron+station+temperature+controlled	$35–$65	☐ MUST for soldering
10	**Solder wire** (60/40 or 63/37, 0.8mm diameter, rosin core)	https://www.amazon.ca/s?k=solder+wire+0.8mm+rosin+core	$10–$18	☐ MUST
11	**Flux pen** (no-clean flux)	https://www.amazon.ca/s?k=no+clean+flux+pen+electronics	$8–$15	☐ HIGH
12	**Brass tip cleaner** (coil cleaner, not wet sponge)	https://www.amazon.ca/s?k=brass+tip+cleaner+soldering	$8–$12	☐ HIGH
13	**Helping hands / PCB holder**	https://www.amazon.ca/s?k=helping+hands+soldering+pcb+holder	$12–$20	☐ HIGH

Phase 3 Total: ~$73–$130 CAD

> ⚠ If you have never soldered before: watch at least 2 YouTube videos on "how to solder pin headers" before touching the iron.

PHASE 4 — Components + Passive Kit (Buy in Week 14–16)

#	Item	Amazon.ca Search	Est. Price (CAD)	Priority
14	**Resistor kit** (1/4W, 0-1MΩ, 600+ pieces)	https://www.amazon.ca/s?k=resistor+assortment+kit	$10–$18	☐ HIGH
15	**Ceramic capacitor kit** (10pF–100nF)	https://www.amazon.ca/s?k=ceramic+capacitor+assortment+kit	$10–$15	☐ HIGH
16	**4.7kΩ resistors (10-pack)** — for I2C pull-ups	https://www.amazon.ca/s?k=4.7k+ohm+resistor	$5–$8	☐ HIGH
17	**LED assortment (5mm)** — for GPIO visual debugging	https://www.amazon.ca/s?k=led+assortment+5mm	$7–$12	☐ OPTIONAL
18	**Push button tactile switches (20-pack)**	https://www.amazon.ca/s?k=tactile+push+button+switch	$6–$10	☐ OPTIONAL

Phase 4 Total: ~$38–$63 CAD

PHASE 5 — Data Logger Upgrades (Buy in Week 16–18)

#	Item	Amazon.ca Search	Est. Price (CAD)	Priority
19	**MicroSD card module (SPI)**	https://www.amazon.ca/s?k=micro+sd+card+module+spi	$8–$15	☐ HIGH (Week 18 feature)
20	**MicroSD card (32GB Class 10)**	https://www.amazon.ca/s?k=micro+sd+card+32gb+class+10	$10–$18	☐ HIGH
21	**DS3231 RTC module** (real-time clock, I2C) — for timestamps in logs	https://www.amazon.ca/s?k=ds3231+rtc+module	$8–$15	☐ OPTIONAL

Phase 5 Total: ~$26–$48 CAD

PHASE 6 — Second Pico + Expansion (Buy in Week 20–24, if budget allows)

#	Item	Amazon.ca Search	Est. Price (CAD)	Priority
22	**Raspberry Pi Pico W** (Wi-Fi model)	https://www.amazon.ca/s?k=raspberry+pi+pico+w	$12–$18	☐ OPTIONAL
23	**Second BME280 or MPU-6050** — for multi-sensor data logger	https://www.amazon.ca/s?k=bme280+breakout	$10–$18	☐ OPTIONAL
24	**0.96" OLED display (I2C, SSD1306)**	https://www.amazon.ca/s?k=oled+display+i2c+ssd1306	$8–$15	☐ OPTIONAL
25	**Male-to-female GPIO ribbon cable (for Pico)**	https://www.amazon.ca/s?k=gpio+ribbon+cable+male+female	$8–$12	☐ OPTIONAL

Phase 6 Total: ~$38–$63 CAD

BOOKS — Buy or Borrow (Physical or eBook)

#	Book	Where to Get	Est. Cost	Priority
B1	**"Writing Science" — Joshua Schimel**	Amazon.ca / library	$30–$45 new, $15–$25 used	☐ MUST (Week 13+)
B2	**"Make: Getting Started with Raspberry Pi Pico"**	Amazon.ca or Maker Press free PDF	Free PDF: https://makerpublishing.com	☐ HIGH
B3	**"SystemVerilog for Verification" — Spear + Tumbush**	Library or eBook	$80–$120 new	☐ OPTIONAL (post-Week 34)

#	Book	Where to Get	Est. Cost	Priority
B4	**"The C Programming Language" — Kernighan + Ritchie (K&R)**	Amazon.ca	$40–$60	□ HIGH (Year 2+)

SUMMARY: Total Cost by Phase

Phase	When to Buy	Approx Cost (CAD)	Status
Phase 0 — Software	Week 1	**$0**	Free
Phase 1 — Core Hardware	Week 4–5	**$49–$77**	□ MUST
Phase 2 — Debug Tools	Week 9–10	**$33–$60**	□ MUST
Phase 3 — Soldering Kit	Week 13–14	**$73–$130**	□ MUST
Phase 4 — Components	Week 14–16	**$38–$63**	□ HIGH
Phase 5 — Data Logger Upgrades	Week 16–18	**$26–$48**	□ HIGH
Phase 6 — Expansion	Week 20–24	**$38–$63**	□ OPTIONAL
Books	Various	**$30–$105**	□ B1 MUST
TOTAL (MUST only)		**~$185–$330 CAD**	
TOTAL (MUST + HIGH)		**~$249–$438 CAD**	
TOTAL (everything)		**~$320–$566 CAD**	

> □ **Budget tip:** Phase 1 + 2 is ~$82–$137. That's enough to start everything through Week 14. Do NOT buy Phase 3+ until you've proven you'll use it. Shop during Amazon Canada sales events (Prime Day, Black Friday) — logic analyzers and kits often go on sale.

TOOLS + SOFTWARE: Zero-Cost Stack (Complete List)

Tool	Purpose	Link
iverilog	Verilog simulation	`sudo apt install iverilog`
GTKWave	Waveform viewer	`sudo apt install gtkwave`
Verilator	Fast RTL simulation	https://www.verilator.org/
VS Code	Editor (Windows + WSL2)	https://code.visualstudio.com/
Python 3.11+	Scripting, parsers, plots	https://www.python.org/
matplotlib	Result figures	`pip install matplotlib`
Zotero	Reference manager	https://www.zotero.org/
PulseView/sigrok	Logic analyzer software	https://sigrok.org/wiki/PulseView
GitHub Actions	CI/CD for your repos	https://github.com/features/actions
cppcheck	C static analysis	`sudo apt install cppcheck`
clang-format	C code formatting	`sudo apt install clang-format`
valgrind	C memory error detection	`sudo apt install valgrind`
Excalidraw	Architecture diagrams	https://excalidraw.com/
minicom	Serial monitor (WSL2)	`sudo apt install minicom`
LaTeX (TeX Live)	Paper formatting	`sudo apt install texlive-full`

WEEKLY PURCHASE DECISION GUIDE

Use this table every Sunday to decide if you should order something:

Week	Buying Decision
1–3	Buy nothing. Install software only.
4	Order Phase 1 NOW (allow 3–7 days shipping)
6	Phase 1 should have arrived — start using hardware
9	Order Phase 2 (logic analyzer + multimeter)
13	Order Phase 3 (soldering station) + Phase 4 (components)
16	Order Phase 5 (SD card module + RTC)
20	Evaluate Phase 6 only if budget allows and you have a clear use case
24+	No new hardware unless RA lab provides it

End of ELITE Roadmap continuation + Shopping Sheet Commit this file to: `embedded-journey/embedded_plan_regina_ELITE_v1_continued.md`